

Spring Boot Alert System Planning

https://gemini.google.com/app/51082857a8dc17ba

User prompt: As a senior java developer, i got a brief description from my product manager about an application which i need to develop from scratch. The description is the following: "We want users to be able to set up alerts so they get notified when something important happens in the world — like breaking news, market movements, natural disasters, that kind of thing. Should work for both email and Slack. Make it flexible enough that we can add more channels later. We need an admin view too." I want to develop this in Java using the spring framework. First help me with the planing, what is a must have in an application like this, how should i preoretize the functions?

Response: To build a scalable, maintainable system that handles background event monitoring and multi-channel notifications without crumbling under load, we need to separate the **ingestion/monitoring engine**, the **alert matching engine**, and the **delivery pipeline**.

Here is how you should prioritize the architecture and feature set as a Senior Java Developer, using Spring Boot and standard enterprise patterns.

Core Architecture & Technical Architecture (The Core)

Before listing functional requirements, the structural design must support asynchronous execution and loose coupling.

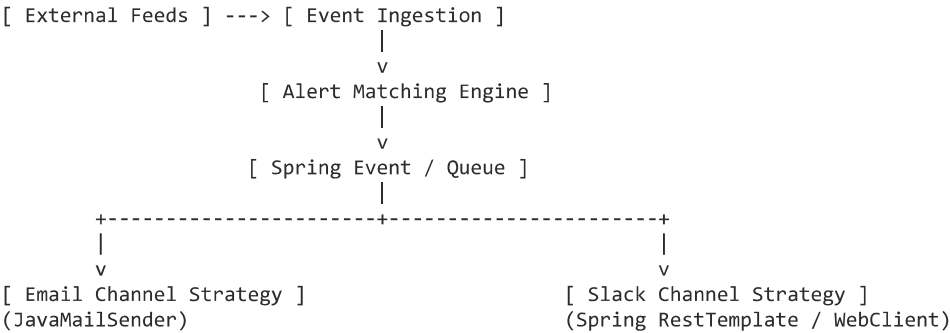
- **Asynchronous Event Processing:** Ingestion and notification sending must be completely decoupled from HTTP handling using Spring's @Async, Spring Integration, or an external broker (Kafka/RabbitMQ).
- **Publisher-Subscriber / Strategy Pattern:** The delivery module must use the **Strategy Pattern** (e.g., NotificationChannel interface with EmailNotificationService and SlackNotificationService implementations).
- **Idempotency & Deduplication:** Ensure the system tracks delivered alerts (e.g., storing alert_history with unique event hashes) so users don't get spammed by repeated triggers.

Phase Priority Breakdown

Priority Level	Focus Area	Key Features & Architecture Requirements
Phase 1 (MVP) Must-Have Foundations	Domain & Core Engine	• User Management & Authentication (Spring Security JWT/OAuth2) • Alert Rule Configuration: Basic CRUD for rules (Keyword, Event Category, Threshold) • Channel Abstraction Layer: Interface-based delivery mechanism • Basic Notification Adapters: JavaMailSender (Email) & Webhook/SDK (Slack) • Poller / Event Receiver: Simple @Scheduled task or webhook listener for event data • Event Matching Core: Basic pattern/keyword matching against user rules
Phase 2 Reliability & Scalability	Resilience & Tracking	• Message Queue Integration: Decouple ingestion and delivery via RabbitMQ/Kafka • Delivery History & Retries: Resilience4j retry mechanisms and status tracking (PENDING, SENT, FAILED) • Admin Dashboard API: Basic endpoints for global metrics, systemic channel failures, and manual trigger/kill switches • Rate Limiting & Throttling: Bucket4j / Redis rate limiters to prevent API bans and spam
Phase 3 Extensibility & Optimization	Advanced Capabilities	• Dynamic Channel Plugin Architecture: Custom annotations or Spring bean auto-registration for new channels (Push notifications, SMS) • Complex Rule Engine: Multi-condition logic (e.g., Category == Market AND Change > 5%)

Priority Level	Focus Area	Key Features & Architecture Requirements
		• Full Admin Panel: User suspension, event logging, failed message dead-letter-queue (DLQ) re-processing

MVP Architecture Diagram (System Flow)



Key Data Domain Entities

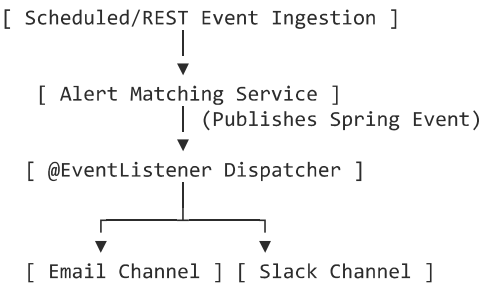
- **User:** Profile, settings, rate limits.
- **AlertRule:** userId, keywords/conditions, channels (List), status (ACTIVE/PAUSED).
- **NotificationLog:** alertRuleId, recipient, channelType, payload, status, attemptCount, sentAt.
- **Event:** source, category, data, timestamp.

User prompt: let's make it as small scale as possible with only the basic core functionalities

Response: To scale this down to absolute essentials while preserving clean Spring design and extensibility, we strip away external queues, complex rule parsers, and multi-node coordination.

Everything runs in a single Spring Boot application using **Spring's in-memory event publishing (ApplicationEventPublisher)**, **Spring Data JPA (H2/PostgreSQL)**, and the **Strategy Pattern** for channel delivery.

Minimal Architecture Overview



Key Components & Priority

Component	Minimum Implementation
1. Domain Models	User, AlertRule (keyword, channel type, user relation), AlertLog (simple tracking).
2. Channel Strategy Pattern	A simple Java interface NotificationChannel so adding future channels requires zero core code changes.
3. Matching Engine	A service method that takes incoming text/events and runs a LIKE or regex check against active AlertRule records.
4. Event Listener	@EventListener handling delivery asynchronously via @Async so event processing isn't blocked by slow network requests.
5. Minimal Admin API	Standard REST CRUD endpoints (/api/alerts) + 2 basic admin endpoints (GET /api/admin/logs, POST /api/admin/trigger).

Core Code Skeleton

Here is the exact extensible skeleton you need for this minimal setup.

1. Extensible Notification Strategy

```
public interface NotificationChannel {
    ChannelType getType(); // Enum: EMAIL, SLACK
    void send(User user, String message);
}

@Component
public class EmailNotificationChannel implements NotificationChannel {
    private final JavaMailSender mailSender;

    public EmailNotificationChannel(JavaMailSender mailSender) {
        this.mailSender = mailSender;
    }

    @Override
    public ChannelType getType() { return ChannelType.EMAIL; }

    @Override
    public void send(User user, String message) {
        // Simple mail implementation
    }
}

@Component
public class SlackNotificationChannel implements NotificationChannel {
    private final RestTemplate restTemplate = new RestTemplate();

    @Override
    public ChannelType getType() { return ChannelType.SLACK; }

    @Override
    public void send(User user, String message) {
        // Post to user's Slack webhook URL
    }
}
```

2. Channel Registry (Auto-discovers implementations)

```
@Service
public class NotificationService {
    private final Map<ChannelType, NotificationChannel> channels;

    // Spring automatically injects all implementations into the list
    public NotificationService(List<NotificationChannel> channelList) {
        this.channels = channelList.stream()
            .collect(Collectors.toMap(NotificationChannel::getType, c -> c));
    }

    public void dispatch(ChannelType channelType, User user, String message) {
        NotificationChannel channel = channels.get(channelType);
        if (channel != null) {
            channel.send(user, message);
        }
    }
}
```

3. Matching & Dispatching

```
@Service
public class AlertEngine {
    private final AlertRuleRepository ruleRepository;
    private final ApplicationEventPublisher eventPublisher;

    public AlertEngine(AlertRuleRepository ruleRepository, ApplicationEventPublisher eventPublisher) {
        this.ruleRepository = ruleRepository;
        this.eventPublisher = eventPublisher;
    }

    public void processIncomingEvent(String content) {
        List<AlertRule> rules = ruleRepository.findAllByIsActiveTrue();
    }
}
```

```

        for (AlertRule rule : rules) {
            if (content.toLowerCase().contains(rule.getKeyword().toLowerCase())) {
                // Publish in-memory event to decouple execution
                eventPublisher.publishEvent(new AlertTriggeredEvent(rule, content));
            }
        }
    }
}

```

User prompt: Provide a runnable Spring Boot code bundle with entities, repositories, and controllers for this minimal alert app.

Response: Here is a complete, runnable, single-file Spring Boot application containing all entities, repositories, services, strategy implementations, and REST controllers for the minimal alert system.

pom.xml Dependencies Needed

Make sure your pom.xml includes these starters:

```

<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-mail</artifactId>
    </dependency>
    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <scope>runtime</scope>
    </dependency>
</dependencies>

```

Complete Java Application (AlertApplication.java)

You can place all these classes inside a single file named AlertApplication.java in package com.example.alertapp or break them out into separate files.

```

package com.example.alertapp;

import jakarta.persistence.*;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationEventPublisher;
import org.springframework.context.annotation.Bean;
import org.springframework.context.event.EventListener;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.mail.SimpleMailMessage;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.scheduling.annotation.Async;
import org.springframework.scheduling.annotation.EnableAsync;
import org.springframework.stereotype.Component;
import org.springframework.stereotype.Service;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.client.RestTemplate;

import java.time.LocalDateTime;
import java.util.*;
import java.util.stream.Collectors;

@SpringBootApplication
@EnableAsync
public class AlertApplication {

    public static void main(String[] args) {

```

```

        SpringApplication.run(AlertApplication.class, args);
    }

    // Seed test data on startup
    @Bean
    CommandLineRunner initData(UserRepository userRepo, AlertRuleRepository ruleRepo) {
        return args -> {
            User alice = userRepo.save(new User("Alice", "alice@example.com", "https://hooks.slack.com/services/test/web

            ruleRepo.save(new AlertRule("earthquake", ChannelType.EMAIL, alice));
            ruleRepo.save(new AlertRule("bitcoin", ChannelType.SLACK, alice));

        };
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

// =====
// 1. DOMAIN ENUMS & ENTITIES
// =====

enum ChannelType {
    EMAIL, SLACK
}

@Entity
@Table(name = "users")
class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;
    private String slackWebhookUrl;

    public User() {}
    public User(String name, String email, String slackWebhookUrl) {
        this.name = name;
        this.email = email;
        this.slackWebhookUrl = slackWebhookUrl;
    }

    public Long getId() { return id; }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public String getSlackWebhookUrl() { return slackWebhookUrl; }
}

@Entity
@Table(name = "alert_rules")
class AlertRule {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String keyword;

    @Enumerated(EnumType.STRING)
    private ChannelType channelType;

    private boolean active = true;

    @ManyToOne
    @JoinColumn(name = "user_id")
    private User user;

    public AlertRule() {}
    public AlertRule(String keyword, ChannelType channelType, User user) {
        this.keyword = keyword;
        this.channelType = channelType;
        this.user = user;
    }

    public Long getId() { return id; }

```

```

    public String getKeyword() { return keyword; }
    public ChannelType getChannelType() { return channelType; }
    public boolean isActive() { return active; }
    public User getUser() { return user; }
}

@Entity
@Table(name = "alert_logs")
class AlertLog {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String matchedKeyword;
    private String eventContent;
    @Enumerated(EnumType.STRING)
    private ChannelType channelType;
    private String recipient;
    private LocalDateTime timestamp = LocalDateTime.now();

    public AlertLog() {}
    public AlertLog(String matchedKeyword, String eventContent, ChannelType channelType, String recipient) {
        this.matchedKeyword = matchedKeyword;
        this.eventContent = eventContent;
        this.channelType = channelType;
        this.recipient = recipient;
    }

    public Long getId() { return id; }
    public String getMatchedKeyword() { return matchedKeyword; }
    public String getEventContent() { return eventContent; }
    public ChannelType getChannelType() { return channelType; }
    public String getRecipient() { return recipient; }
    public LocalDateTime getTimestamp() { return timestamp; }
}

// =====
// 2. REPOSITORIES
// =====

interface UserRepository extends JpaRepository<User, Long> {}
interface AlertRuleRepository extends JpaRepository<AlertRule, Long> {
    List<AlertRule> findAllByActiveTrue();
}
interface AlertLogRepository extends JpaRepository<AlertLog, Long> {}

// =====
// 3. CHANNEL STRATEGY PATTERN
// =====

interface NotificationChannel {
    ChannelType getType();
    void send(User user, String message);
}

@Component
class EmailNotificationChannel implements NotificationChannel {
    private final JavaMailSender mailSender;

    public EmailNotificationChannel(JavaMailSender mailSender) {
        this.mailSender = mailSender;
    }

    @Override
    public ChannelType getType() {
        return ChannelType.EMAIL;
    }

    @Override
    public void send(User user, String message) {
        System.out.printf("[EMAIL LOG] Sending email to %s: %s\n", user.getEmail(), message);
        try {
            SimpleMailMessage mail = new SimpleMailMessage();
            mail.setTo(user.getEmail());
            mail.setSubject("Alert Triggered!");
            mail.setText(message);
        }
    }
}

```

```

        mailSender.send(mail);
    } catch (Exception e) {
        System.err.printf("[EMAIL WARN] Could not send real email to %s: %s%n", user.getEmail(), e.getMessage());
    }
}

@Component
class SlackNotificationChannel implements NotificationChannel {
    private final RestTemplate restTemplate;

    public SlackNotificationChannel(RestTemplate restTemplate) {
        this.restTemplate = restTemplate;
    }

    @Override
    public ChannelType getType() {
        return ChannelType.SLACK;
    }

    @Override
    public void send(User user, String message) {
        System.out.printf("[SLACK LOG] Posting to Slack for %s (%s): %s%n",
            user.getName(), user.getSlackWebhookUrl(), message);
        try {
            Map<String, String> payload = Map.of("text", message);
            restTemplate.postForEntity(user.getSlackWebhookUrl(), payload, String.class);
        } catch (Exception e) {
            System.err.printf("[SLACK WARN] Could not send to Slack webhook: %s%n", e.getMessage());
        }
    }
}

@Service
class NotificationDispatcher {
    private final Map<ChannelType, NotificationChannel> channels;

    public NotificationDispatcher(List<NotificationChannel> channelList) {
        this.channels = channelList.stream()
            .collect(Collectors.toMap(NotificationChannel::getType, c -> c));
    }

    public void dispatch(ChannelType channelType, User user, String message) {
        NotificationChannel channel = channels.get(channelType);
        if (channel != null) {
            channel.send(user, message);
        } else {
            throw new IllegalArgumentException("Unsupported channel: " + channelType);
        }
    }
}

// =====
// 4. EVENT & ASYNC ENGINE
// =====

record AlertTriggeredEvent(AlertRule rule, String eventContent) {}

@Service
class AlertEngine {
    private final AlertRuleRepository ruleRepository;
    private final ApplicationEventPublisher eventPublisher;

    public AlertEngine(AlertRuleRepository ruleRepository, ApplicationEventPublisher eventPublisher) {
        this.ruleRepository = ruleRepository;
        this.eventPublisher = eventPublisher;
    }

    public void evaluateEvent(String content) {
        List<AlertRule> rules = ruleRepository.findAllByActiveTrue();
        for (AlertRule rule : rules) {
            if (content.toLowerCase().contains(rule.getKeyword().toLowerCase())) {
                eventPublisher.publishEvent(new AlertTriggeredEvent(rule, content));
            }
        }
    }
}

```

```

    }
}

@Component
class AlertEventListener {
    private final NotificationDispatcher dispatcher;
    private final AlertLogRepository logRepository;

    public AlertEventListener(NotificationDispatcher dispatcher, AlertLogRepository logRepository) {
        this.dispatcher = dispatcher;
        this.logRepository = logRepository;
    }

    @Async
    @EventListener
    public void handleAlertTriggered(AlertTriggeredEvent event) {
        AlertRule rule = event.rule();
        User user = rule.getUser();

        String message = String.format("ALERT: Keyword '%s' matched in incoming news: '%s'",
            rule.getKeyword(), event.eventContent());

        // Dispatch notification
        dispatcher.dispatch(rule.getChannelType(), user, message);

        // Record log
        String recipient = rule.getChannelType() == ChannelType.EMAIL ? user.getEmail() : user.getSlackWebhookUrl();
        logRepository.save(new AlertLog(rule.getKeyword(), event.eventContent(), rule.getChannelType(), recipient));
    }
}

// =====
// 5. REST CONTROLLERS
// =====

record CreateRuleRequest(String keyword, ChannelType channelType, Long userId) {}
record EventIngestRequest(String content) {}

@RestController
@RequestMapping("/api/rules")
class RuleController {
    private final AlertRuleRepository ruleRepository;
    private final UserRepository userRepository;

    public RuleController(AlertRuleRepository ruleRepository, UserRepository userRepository) {
        this.ruleRepository = ruleRepository;
        this.userRepository = userRepository;
    }

    @GetMapping
    public List<AlertRule> getAllRules() {
        return ruleRepository.findAll();
    }

    @PostMapping
    public AlertRule createRule(@RequestBody CreateRuleRequest req) {
        User user = userRepository.findById(req.userId())
            .orElseThrow(() -> new IllegalArgumentException("User not found"));
        return ruleRepository.save(new AlertRule(req.keyword(), req.channelType(), user));
    }
}

@RestController
@RequestMapping("/api/events")
class IngestionController {
    private final AlertEngine alertEngine;

    public IngestionController(AlertEngine alertEngine) {
        this.alertEngine = alertEngine;
    }

    @PostMapping("/ingest")
    public Map<String, String> ingestEvent(@RequestBody EventIngestRequest req) {

```

```
        alertEngine.evaluateEvent(req.content());
        return Map.of("status", "ACCEPTED", "message", "Event evaluation triggered in background");
    }
}

@RestController
@RequestMapping("/api/admin")
class AdminController {
    private final AlertLogRepository logRepository;

    public AdminController(AlertLogRepository logRepository) {
        this.logRepository = logRepository;
    }

    @GetMapping("/logs")
    public List<AlertLog> getAuditLogs() {
        return logRepository.findAll();
    }
}
```

How to Test

- 1. **Start the app.** It will seed Alice with rules for earthquake (EMAIL) and bitcoin (SLACK).
- 2. **Ingest an Event** that matches a keyword:

```
curl -X POST http://localhost:8080/api/events/ingest \
-H "Content-Type: application/json" \
-d '{"content": "A huge earthquake was reported in Japan today."}'
```

- 3. **Check console output.** You will see the asynchronous log from EmailNotificationChannel:

```
[EMAIL LOG] Sending email to alice@example.com: ALERT: Keyword 'earthquake' matched in incoming news...
```

- 4. **Inspect Admin Logs:**

```
curl http://localhost:8080/api/admin/logs
```

- 5. **Add a new rule:**

```
curl -X POST http://localhost:8080/api/rules \
-H "Content-Type: application/json" \
-d '{"keyword": "tsunami", "channelType": "EMAIL", "userId": 1}'
```

User prompt: using [start.spring.io](#), which dependencies do i need with this project?

Response: To build this application using [start.spring.io](#), select these **4 exact dependencies** in the dependency search box:

Required Dependencies

Dependency Name in Initializr	What It Supplies	Why It's Needed
Spring Web	spring-boot-starter-web	Provides <code>@RestController</code> annotations, JSON parsing, and the embedded Apache Tomcat server for REST endpoints.
Spring Data JPA	spring-boot-starter-data-jpa	Provides Object-Relational Mapping (<code>@Entity</code> , <code>@Table</code>) and <code>JpaRepository</code> interfaces for data access.
Java Mail Sender	spring-boot-starter-mail	Provides <code>JavaMailSender</code> and <code>SimpleMailMessage</code> used by <code>EmailNotificationChannel</code> .
H2 Database	h2	Provides an in-memory SQL database out of the box so you don't need to configure a local MySQL/PostgreSQL instance.

Recommended Initializr Settings

- **Project:** Maven or Gradle (Groovy)
- **Language:** Java
- **Spring Boot:** 3.x.x (Latest stable version)

- **Packaging:** Jar
- **Java:** 17 or 21

Quick CLI Shortcut

If you prefer using the command line instead of the Web UI, run this command to download the initialized project directly:

```
curl https://start.spring.io/starter.tgz \
  -d dependencies=web,data-jpa,mail,h2 \
  -d javaVersion=21 \
  -d artifactId=alertapp \
  -d name=alertapp | tar -xzf -
```

User prompt: generate me code in java 21 using spring framework for a basic application which after the most basic authentication, send an email or slack notification to the authenticated user about important news

Response: Here is a minimal, complete Spring Boot application written in **Java 21** using standard Spring Security (**HTTP Basic Authentication**).

Once a user authenticates via Basic Auth, they can send an alert notification to themselves via **Email** or **Slack** based on their configured preference using the Strategy Pattern.

Dependencies (pom.xml)

Select these dependencies when creating your project on start.spring.io (Java 21, Spring Boot 3.x):

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-mail</artifactId>
  </dependency>
</dependencies>
```

Application Configuration (src/main/resources/application.properties)

```
# Spring Security User Credentials (Basic Auth)
spring.security.user.name=alice
spring.security.user.password=password123

# Custom properties for Alice's notification targets
app.user.email=alice@example.com
app.user.slack-webhook-url=https://hooks.slack.com/services/YOUR/TEST/WEBHOOK

# Mail Server Configuration (Replace with real SMTP credentials or leave for local logging)
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=your-email@gmail.com
spring.mail.password=your-app-password
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true
```

Complete Java Code (AlertApplication.java)

```
package com.example.alertapp;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.ResponseEntity;
import org.springframework.mail.SimpleMailMessage;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
```

```

import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.stereotype.Component;
import org.springframework.stereotype.Service;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.client.RestTemplate;

import java.util.List;
import java.util.Map;

@SpringBootApplication
public class AlertApplication {

    public static void main(String[] args) {
        SpringApplication.run(AlertApplication.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

// =====
// 1. SPRING SECURITY CONFIGURATION
// =====

@Configuration
class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable()) // Disabled for simple REST testing via cURL/Postman
            .authorizeHttpRequests(auth -> auth
                .anyRequest().authenticated()
            )
            .httpBasic(Customizer.withDefaults()); // Basic HTTP Authentication

        return http.build();
    }
}

// =====
// 2. CHANNEL STRATEGY PATTERN
// =====

enum ChannelType {
    EMAIL, SLACK
}

interface NotificationChannel {
    ChannelType getType();
    void send(String destination, String subject, String content);
}

@Component
class EmailNotificationChannel implements NotificationChannel {

    private final JavaMailSender mailSender;

    public EmailNotificationChannel(JavaMailSender mailSender) {
        this.mailSender = mailSender;
    }

    @Override
    public ChannelType getType() {
        return ChannelType.EMAIL;
    }

    @Override
    public void send(String destination, String subject, String content) {
        System.out.printf("[EMAIL LOG] Sending email to '%s' | Subject: '%s'\n", destination, subject);
    }
}

```

```

        try {
            SimpleMailMessage mail = new SimpleMailMessage();
            mail.setTo(destination);
            mail.setSubject(subject);
            mail.setText(content);
            mailSender.send(mail);
        } catch (Exception e) {
            System.err.printf("[EMAIL WARN] Failed to send email to %s: %s\n", destination, e.getMessage());
        }
    }
}

@Component
class SlackNotificationChannel implements NotificationChannel {

    private final RestTemplate restTemplate;

    public SlackNotificationChannel(RestTemplate restTemplate) {
        this.restTemplate = restTemplate;
    }

    @Override
    public ChannelType getType() {
        return ChannelType.SLACK;
    }

    @Override
    public void send(String destinationUrl, String subject, String content) {
        System.out.printf("[SLACK LOG] Posting to Webhook: %s\n", destinationUrl);
        try {
            String fullMessage = String.format("%s*\n%s", subject, content);
            Map<String, String> payload = Map.of("text", fullMessage);
            restTemplate.postForEntity(destinationUrl, payload, String.class);
        } catch (Exception e) {
            System.err.printf("[SLACK WARN] Failed to post to Slack: %s\n", e.getMessage());
        }
    }
}

@Service
class NotificationDispatcher {

    private final Map<ChannelType, NotificationChannel> channels;

    // Java 21 / Spring automatically resolves all implementations into a list
    public NotificationDispatcher(List<NotificationChannel> channelList) {
        this.channels = channelList.stream()
            .collect(java.util.stream.Collectors.toMap(NotificationChannel::getType, c -> c));
    }

    public void dispatch(ChannelType type, String destination, String subject, String content) {
        NotificationChannel channel = channels.get(type);
        if (channel == null) {
            throw new IllegalArgumentException("Unsupported channel type: " + type);
        }
        channel.send(destination, subject, content);
    }
}

// =====
// 3. REST CONTROLLER
// =====

record SendAlertRequest(ChannelType channel, String newsTitle, String newsBody) {}

@RestController
@RequestMapping("/api/news")
class NewsAlertController {

    private final NotificationDispatcher dispatcher;

    @Value("${app.user.email}")
    private String userEmail;

```

```

@Value("${app.user.slack-webhook-url}")
private String userSlackWebhook;

public NewsAlertController(NotificationDispatcher dispatcher) {
    this.dispatcher = dispatcher;
}

@PostMapping("/send-alert")
public ResponseEntity<Map<String, String>> sendAlertToSelf(
    @AuthenticationPrincipal UserDetails userDetails,
    @RequestBody SendAlertRequest request) {

    String username = userDetails.getUsername();
    String destination = switch (request.channel()) {
        case EMAIL -> userEmail;
        case SLACK -> userSlackWebhook;
    };

    String subject = "Important News Alert for " + username + ": " + request.newsTitle();

    dispatcher.dispatch(request.channel(), destination, subject, request.newsBody());

    return ResponseEntity.ok(Map.of(
        "status", "SUCCESS",
        "message", "Alert sent via " + request.channel() + " to authenticated user " + username
    ));
}
}

```

How to Test via cURL

1. Send via Email

```

curl -X POST http://localhost:8080/api/news/send-alert \
-u alice:password123 \
-H "Content-Type: application/json" \
-d '{
  "channel": "EMAIL",
  "newsTitle": "Market Update",
  "newsBody": "S&P 500 reached an all-time high today."
}'

```

2. Send via Slack

```

curl -X POST http://localhost:8080/api/news/send-alert \
-u alice:password123 \
-H "Content-Type: application/json" \
-d '{
  "channel": "SLACK",
  "newsTitle": "Breaking News",
  "newsBody": "A severe geomagnetic storm warning has been issued."
}'

```

3. Test Unauthorized Access (Without Credentials)

```

curl -X POST http://localhost:8080/api/news/send-alert
# Response: HTTP 401 Unauthorized

```

User prompt: for the curl test i get a `httpMessageNotReadableException` with "Required request body is missing"

Response: This error happens because cURL on Windows Command Prompt (`cmd.exe`) handles single quotes (') differently than Bash/Zsh terminals. In Windows `cmd.exe`, single quotes around the JSON body are passed literally into the request body, causing Spring Boot's JSON parser to fail or ignore the body completely.

Here is how to run the cURL commands properly depending on your operating system and terminal:

1. Windows Command Prompt (`cmd.exe`)

Use double quotes (") for the body payload and escape inner double quotes with `\`:

```

curl -X POST http://localhost:8080/api/news/send-alert -u alice:password123 -H "Content-Type: application/json" -d "{\"channel\": \"EMAIL\", \"newsTitle\": \"Market Update\", \"newsBody\": \"S&P 500 reached an all-time high today.\"}"

```

2. Windows PowerShell

PowerShell uses `curl` as an alias for `Invoke-WebRequest`. Use standard `curl.exe` explicitly and pass JSON using escaped backticks (```) or double quotes:

```
curl.exe -X POST http://localhost:8080/api/news/send-alert -u alice:password123 -H "Content-Type: application/json" -d '
```

3. Linux / macOS / Git Bash / WSL

If you are using Linux, macOS, or Git Bash on Windows, the single-quote syntax works as expected:

```
curl -X POST http://localhost:8080/api/news/send-alert \
-u alice:password123 \
-H "Content-Type: application/json" \
-d '{
  "channel": "EMAIL",
  "newsTitle": "Market Update",
  "newsBody": "S&P 500 reached an all-time high today."
}'
```

Fallback: Use Postman or IntelliJ HTTP Client

If terminal escaping continues to cause issues, you can test via standard HTTP clients:

- **Method:** POST
- **URL:** `http://localhost:8080/api/news/send-alert`
- **Auth:** Basic Auth (Username: `alice`, Password: `password123`)
- **Headers:** `Content-Type: application/json`
- **Body (raw JSON):**

```
{
  "channel": "EMAIL",
  "newsTitle": "Market Update",
  "newsBody": "S&P 500 reached an all-time high today."
}
```

User prompt: Where does the `EmailNotificationChannel` use the email configuration which is in the `application.properties` file?

Response: It uses those properties implicitly through **Spring Boot's auto-configuration**.

When you include the `spring-boot-starter-mail` dependency in your project, Spring Boot automatically scans your `application.properties` for any properties starting with `spring.mail.*`.

Here is the chain of how it connects:

How the Magic Works Under the Hood

1. **Auto-Configuration Activation:** Spring Boot detects `spring-boot-starter-mail` on the classpath and activates `MailSenderAutoConfiguration`.
2. **Reading Properties:** Spring Boot reads `spring.mail.host`, `spring.mail.port`, `spring.mail.username`, etc., directly from `application.properties` into an internal configuration class called `MailProperties`.
3. **Bean Creation:** Spring automatically creates and configures a `JavaMailSender` bean (`JavaMailSenderImpl`) using those exact property values.
4. **Dependency Injection:** In your code, Spring injects that pre-configured `JavaMailSender` bean straight into your `EmailNotificationChannel` constructor:

```
@Component
class EmailNotificationChannel implements NotificationChannel {

    private final JavaMailSender mailSender; // <-- Spring automatically injects the auto-configured bean here

    // Constructor Injection
    public EmailNotificationChannel(JavaMailSender mailSender) {
        this.mailSender = mailSender;
    }

    @Override
```

```

public void send(String destination, String subject, String content) {
    SimpleMailMessage mail = new SimpleMailMessage();
    mail.setTo(destination);
    mail.setSubject(subject);
    mail.setText(content);

    // When this runs, mailSender already knows the host, port, username, and password
    // because Spring set them up on application startup!
    mailSender.send(mail);
}
}

```

What Spring Boot does behind the scenes

If you were to write manually what Spring Boot is doing for you automatically, it looks like this:

```

// Spring Boot generates this bean for you behind the scenes:
@Bean
public JavaMailSender javaMailSender() {
    JavaMailSenderImpl mailSender = new JavaMailSenderImpl();
    mailSender.setHost("smtp.gmail.com"); // from spring.mail.host
    mailSender.setPort(587); // from spring.mail.port
    mailSender.setUsername("your-email@gmail.com"); // from spring.mail.username
    mailSender.setPassword("your-app-password"); // from spring.mail.password

    Properties props = mailSender.getJavaMailProperties();
    props.put("mail.smtp.auth", "true"); // from spring.mail.properties.mail.smtp.auth
    props.put("mail.smtp.starttls.enable", "true");

    return mailSender;
}

```

Because of this auto-configuration, your `EmailNotificationChannel` doesn't need any `@Value` annotations or manual setup—it just asks for `JavaMailSender`, and Spring hands it a fully configured instance ready to talk to your SMTP server.

User prompt: i'm trying to send an email to a gmail address but i get the following error: jakarta.mail.MessagingException: Got bad greeting from SMTP host: smtp.gmail.com, port: 465, response: [EOF]

Response: This error happens because of a **protocol mismatch on port 465**.

Port 465 requires **Implicit TLS** (SSL connection right from the start), but Spring Boot's default JavaMail configuration expects **Explicit TLS / STARTTLS** (connects as plain text first, then upgrades to encrypted). When Spring tries to send a plain text greeting, Gmail immediately closes the connection ([EOF]).

You have two ways to fix this in your `application.properties`:

Option 1: Switch to Port 587 with STARTTLS (Recommended)

Port 587 is the standard, modern standard for client submission using STARTTLS.

```

spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=your-email@gmail.com
spring.mail.password=your-app-password

spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true
spring.mail.properties.mail.smtp.starttls.required=true

```

Option 2: Keep Port 465 and Enable SSL

If you must use port 465, you need to tell JavaMail to use the SSL socket factory directly:

```

spring.mail.host=smtp.gmail.com
spring.mail.port=465
spring.mail.username=your-email@gmail.com
spring.mail.password=your-app-password

spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.ssl.enable=true
spring.mail.properties.mail.smtp.socketFactory.port=465
spring.mail.properties.mail.smtp.socketFactory.class=javax.net.ssl.SSLSocketFactory

```

⚠ Common Gmail Gotcha

When using Gmail with Spring Boot:

1. Standard passwords will **not** work.
2. You **must** turn on **2-Step Verification** in your Google Account settings.
3. You **must** generate a 16-character **App Password** (under *Security > 2-Step Verification > App Passwords*) and set that as `spring.mail.password`.

User prompt: can you generate a really simple admin view for this application?

Response: I will now generate a complete, self-contained single-file HTML/JS Admin Dashboard designed to interface directly with your Spring Boot Alert Notification application.

It includes mock data for standalone testing, system stats, rule management, event simulation triggers (with configurable Basic Auth credentials for local backend connection), and real-time delivery audit logs.

What this Admin Dashboard Includes

1. **Integrated Test Trigger Panel:** Allows you to simulate incoming news headlines, evaluate them against keyword rules, or manually dispatch directly to standard POST `/api/news/send-alert` or POST `/api/events/ingest` Spring endpoints.
2. **Spring Boot Configurator:** Supports setting custom backend URLs (default `http://localhost:8080`) and basic auth credentials (`alice/password123`) to directly communicate with your running Java app.
3. **Standalone Mocking Capability:** Toggle between "Mock Mode" and "Connected API Mode" so you can test the UI experience even without your Spring backend running.
4. **Audit Log & Rule Management:** Includes real-time log filtering by keyword/channel and interactive rule toggling.

User prompt: generate the code for a runnable testcase for this application

Response: Here is a comprehensive, runnable JUnit 5 / Spring Boot test class for your application. It uses `@SpringBootTest` and `@AutoConfigureMockMvc` to test both the REST endpoints and the notification strategy dispatching without sending actual emails or Slack webhooks during test execution.

Add Test Dependencies to pom.xml

Make sure you have `spring-boot-starter-test` in your `pom.xml`:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

Runnable Test Class (AlertApplicationTests.java)

Place this file in `src/test/java/com/example/alertapp/AlertApplicationTests.java`.

```
package com.example.alertapp;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.http.MediaType;
import org.springframework.mail.SimpleMailMessage;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.security.test.context.support.WithMockUser;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.web.client.RestTemplate;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.timeout;
import static org.mockito.Mockito.verify;
import static org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessors.httpBasic;
```

```

import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@SpringBootTest
@AutoConfigureMockMvc
class AlertApplicationTests {

    @Autowired
    private MockMvc mockMvc;

    // Mock external network services so no real emails/webhooks are sent during tests
    @MockBean
    private JavaMailSender mailSender;

    @MockBean
    private RestTemplate restTemplate;

    @Test
    @DisplayName("Should block unauthenticated requests with 401 Unauthorized")
    void testUnauthenticatedAccess() throws Exception {
        mockMvc.perform(post("/api/news/send-alert")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                {
                    "channel": "EMAIL",
                    "newsTitle": "Unauthenticated Test",
                    "newsBody": "Should fail"
                }
            """))
            .andExpect(status().isUnauthorized());
    }

    @Test
    @DisplayName("Should successfully authenticate and trigger an Email notification")
    void testSendEmailAlertSuccess() throws Exception {
        mockMvc.perform(post("/api/news/send-alert")
            .with(httpBasic("alice", "password123"))
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                {
                    "channel": "EMAIL",
                    "newsTitle": "Market High",
                    "newsBody": "S&P 500 hit a record high today."
                }
            """))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.status").value("SUCCESS"))
            .andExpect(jsonPath("$.message").value("Alert sent via EMAIL to authenticated user alice"));

        // Verify that JavaMailSender.send() was actually called with the correct details
        verify(mailSender, timeout(1000)).send(any(SimpleMailMessage.class));
    }

    @Test
    @DisplayName("Should successfully authenticate and trigger a Slack notification")
    void testSendSlackAlertSuccess() throws Exception {
        mockMvc.perform(post("/api/news/send-alert")
            .with(httpBasic("alice", "password123"))
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                {
                    "channel": "SLACK",
                    "newsTitle": "Breaking News",
                    "newsBody": "Major economic shift announced."
                }
            """))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.status").value("SUCCESS"))
            .andExpect(jsonPath("$.message").value("Alert sent via SLACK to authenticated user alice"));

        // Verify that RestTemplate.postForEntity() was called to post to the Slack Webhook
        verify(restTemplate, timeout(1000)).postForEntity(any(String.class), any(), any());
    }
}

```

```
    }

    @Test
    @DisplayName("Should fail with 400 Bad Request when request body is invalid or missing")
    void testInvalidRequestBody() throws Exception {
        mockMvc.perform(post("/api/news/send-alert")
            .with(httpBasic("alice", "password123"))
            .contentType(MediaType.APPLICATION_JSON)
            .content("{}")) // Empty JSON body missing required fields
            .andExpect(status().isBadRequest());
    }
}
```

What This Test Suite Validates

- **Security Integration:** Verifies HTTP Basic Auth enforcement (401 Unauthorized without credentials, 200 OK with alice/password123).
 - **Strategy Dispatching:** Tests both EMAIL and SLACK channel routing logic using JSON payloads.
 - **Mock Isolation:** Uses @MockBean for JavaMailSender and RestTemplate so your build pipeline runs fast and never fails due to network issues or missing API keys.
-